Coding and Testing, part-II

S. S Satapathy

Logic in Testing

xxxx
If(a>b&&b>c&&a>c){
Do this
}
else {
Do that
}
xxxxx
	xxxx
	If(a>b&&b>c&&c>a){
	Do this
	}
	else {
	Do that
	}
	xxxxx

Check if the conditions are OK?

Logic in Testing

xxxx **If(a>b&&b>c&&~~a>c~~){** **Do this** **}** **else {** **Do that** **}** xxxxx	xxxx **If(a>b&&b>c&&c>a){** **Do this** **}** **else {** **Do that** **}** xxxxx

a>c is redundant **c>a is impossible**

Testing based on Covering Predicates

```
void funLeapYear(int Y)
{
        int FLAG;
        if((Y%400==0) || (Y%100!=0 && Y%4==0))
                FLAG=0;
        else
                FLAG=1;
        if(FLAG==0)
                printf("Leap Year");
        else
                printf ("Not Leap Year");
}
```

Truth Tables of Logical Operators

Logical Conjunction (AND)		
p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

Logical Disjunction (OR)		
p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Implication $p \rightarrow q$		
p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

$p \rightarrow q$ is equivalent to $\neg p \vee q$

Logical Equality		
p	q	$p \leftrightarrow q$
T	T	T
T	F	F
F	T	F
F	F	T

p XNOR q (also written as **p ↔ q**, **Epq**, **p = q**, or **p ≡ q**)

Exclusive Disjunction		
p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

p XOR q (also written as **p ⊕ q**, **Jpq**, **p ≠ q**, or **p ↮ q**)

XOR can also be written as $(p \wedge \neg q) \vee (\neg p \wedge q)$.

Predicates and Clauses

- **Every decision statement** in a program, whether it is a **if**, **switch**, **loop**, **contains** a **logical predicate** that is an expression **evaluating** to Boolean value **T/F**.
  - E.g. **((x>y) v C v fun(z)) is a predicate** where **x** and **y** are **non-Boolean variables**, **C** is a **Boolean variable** and **fun(z) returning a Boolean value**. **v** is the **disjunction** operator.
- A **clause is a predicate** that does **not contain any logical operators**.
  - In the above example **((x>y) v C v fun(z)) the logical predicate contains three clauses (x>y), C and fun(z)**
- There can be **different ways of writing a predicate** such that **all of them are equivalent**.

Testing and Covering Predicates

- **Predicates are used in testing** as follows :
  - Developing a **model of the software as one or more predicates**
  - **Testing Requirement** is to **satisfy some combination of clauses**
- Abbreviations used:
  - **P** is the **set of predicates**
  - **p** is a **single predicate** in P
  - **C** is the **set of clauses** in P
  - $\mathbf{C_p}$ is the **set of clauses** in predicate p
  - **c** is a **single clause** in C

Testing based on Covering Predicates

- Abbreviations used:
  - P is the set of predicates
  - p is a single predicate in P
  - C is the set of clauses in P
  - C_p is the set of clauses in predicate p
  - c is a single clause in C

```
if(Y%400==0 || (Y%100!=0&&Y%4==0))
        FLAG=0;
if(FLAG==0)
        printf("Leap Year");
else
        printf ("Not Leap Year");
```

- P={(Y%400==0 || (Y%100!=0&&Y%4==0)), (FLAG==0)}
- C ={(Y%400==0), (Y%100!=0), (Y%4==0), (FLAG==0)}

Predicate coverage

- **Predicate Coverage (PC)**: For each **p** in **P**, **TR** contains two requirements: **p** evaluates to **TRUE** and **p** evaluates to **FALSE**.
- E.g. for the predicate **(x>y) v C v fun(z)**, the tests that satisfy predicate coverage are
  - x=5,y=3, C=TRUE, fun(z)=FALSE (the predicate evaluates to TRUE)
  - x=5,y=8, C=FALSE, fun(z)=FALSE (the predicate evaluates to FALSE)
- For a set of predicates associated with branches/ edges in CFG, **Predicate coverage is same as edge/branch coverage**.

Clause Coverage

- **Individual clauses are not exercised in predicate coverage**.
  - For e.g. in the predicate (x>y) v C v fun(z), **fun(z)** in **both the test cases evaluates to FALSE** in previous slide example.
- **Clause Coverage (CC): For each c in C, TR** contains two requirements: **c evaluates to TRUE** and **c evaluates to FALSE**.
- For e.g., clause coverage in the predicate (x>y) v C v fun(z), needs each clause to be once TRUE and once FALSE. And therefore one of the tests for clause coverage could be:
  - x=5,y=3, C=TRUE, fun(z)=TRUE and
  - x=5,y=8, C=FALSE, fun(z)=FALSE
  - **In this example clause coverage subsumes(or includes) predicate coverage, but not necessarily always**

Predicate vs. Clause Coverage

- **Clause coverage does not subsumes predicate coverage**.
- Consider p=a v b. The clauses are a and b. There are four test inputs considering all combinations of T/F values of a and b

	a	b	a v b
1	T	T	T
2	F	T	T
3	T	F	T
4	F	F	F

- The test set {2,3} above satisfies the clause coverage, but the predicate is true in both the cases. Hence, predicate coverage is not satisfied.

Combinatorial Coverage

- **Combinatorial Coverage (CoC)**: For each p in P, TR contains test requirements for the clauses in C_p to evaluate to each possible combination of truth values.
- It is also called as multiple condition coverage.
- For the predicate p=a v b, combinatorial coverage will test all the values in the truth table given below.

	a	b	a v b
1	T	T	T
2	F	T	T
3	T	F	T
4	F	F	F

Combinatorial Coverage: Feasibility

- A predicate **p** with **n clauses** has **2^n** possible assignments of **truth values**, and therefore it may **not be feasible to satisfy** combinatorial coverage when **n** is **large**, may not be necessary always also.
- What actually needed is **a criteria that capture the effect of each clause** on its predicate and to do this **within a reasonable number of tests**.
  - In our example, **test set {1,4}** makes predicate TRUE once and FALSE once

	a	b	a v b
1	T	T	T
~~2~~	~~F~~	~~T~~	~~T~~
~~3~~	~~T~~	~~F~~	~~T~~
4	F	F	F

A Clause Determining a Predicate

- Clause coverage has a weakness: Truth **value of a clause DO NOT** always **make difference to the value of the whole predicate**.
  - Consider a test case for the clause coverage, which causes each clause to be true: (**5<10**) OR **TRUE** AND (**i>=i*i**). **Only 1st clause matters**
- Research literature provides a lot of idea for reducing number of test cases, but often confusing.
- But the general idea is
- Test each clause independent of other clauses
- **To really test the results of a clause, the clause should be** the **determining factor in the value of the predicate**
- For this we need to understand
  - concept of "**a clause determining a predicate**" and
  - The idea of "**making clauses active**"

Active Clauses

- **Determination:** A **clause c_i** in predicate p, called the a **major clause**, determines p if and only if the remaining **minor clauses** c_j, $j \neq i$, are such that **changing the truth value of c_i changes the truth value of p**.
- This is considered to make the clause c_i **active**.

	a	b	a v b
1	T	T	T
2	T	**F**	T
3	F	T	T
4	F	**F**	F

When **b = F**, value of ***p*** depends on the value of ***a.*** Therefore,
if *b*=F, *a* determines *p*
If *a*=F, *b* determines *p*.

	a	b	a ^ b
1	T	**T**	T
2	T	F	F
3	F	**T**	F
4	F	F	F

When **b = T**, value of ***p*** depends on the value of ***a.*** Therefore,
if *b*=T, *a* determines *p*
If *a*=T, *b* determines *p*.

Clause determining a predicate

- When we are designing test cases our **GOAL** should be to **find tests for each clause when the clause determines the value of the predicate**.
- This is formalized in several criteria that have subtle, but very important differences.
  - They are know as **active clause coverage ACC**

Active Clause coverage

- Active Clause Coverage (ACC): For each *p* in P and **each major clause c_i** in C_p, choose minor clause c_j, $j \neq i$, so that c_i determines p.
- **TR** has two requirements **for each c_i**:
  - c_i evaluates to true and
  - c_i evaluates to false.
- For e.g. if **p=avb**, we get four requirements in TR, two for clause **a** and two for clause **b**. **b** needs to be false for **a** to determine p and vice versa.
- There are **three distinct TR**s above:
  - **{(a=T, b=F), (a=F, b=T), (a=F, b=F)}**

***a* is the major clause**

	a	b	avb
c_i=a	T	F	T
	F	F	F
c_i=b	F	T	T
	F	F	F

***b* is the major clause**

Another Example of ACC

- Truth table for the predicate $p = a \wedge (b \vee c)$

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F
8	F	F	F	F	F

Another Example of ACC

- When **clause *a*** becomes active?

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F
8	F	F	F	F	F

- Choose a row with ***a* evaluates to T** and another row with ***a* evaluates to F**.

Another Example of ACC

- When **clause (*bvc*)** becomes active?

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F
8	F	F	F	F	F

- Choose a row with ***bvc* evaluates to T** and another row with ***bvc* evaluates to F**.

Another Example of ACC

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F

Sl. No.	*a*	*b*	*c*	*b v c*	
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F

- **Rows (1,5,4), (2,6,4), (3,7,4),** …. having **clause a (when it is active) T once and F once**, **clause (bvc) (when it is active) T once and F once** are **the potential solutions**.

Active Clause coverage

- Active Clause Coverage (**ACC**) **chooses different clauses to be major clause in turn**.
- TR has requirements for each clause as a major clause.
- **For a predicate with n clauses, n+1 distinct test requirements are suffice to achieve active clause coverage.**
- **Modified Condition Decision Coverage (MCDC)** criterion is a criterion that states **each clause in a predicate be made true and false in turn** and the predicate be tested for each clause this way.
  - This is same as ACC.
  - **This is mandatory be several certification standards for safety critical software (e.g. s/w used in aviation industry).**

Ambiguity in Active Clause coverage

- In **Active Clause Coverage (ACC)**, the value that **the minor clause** takes is not given any importance.
- The **minor clause could all take the same value or could take different values**.

a **is the major clause**

b **is the major clause**

	a	b	avb
c_i=a	T	F	T
	F	F	F
c_i=b	F	T	T
	F	F	F

In this example, the minor clause has the same values when the major clause is T and F. But we may not have this always as shown in next example ….

Ambiguity in Active Clause coverage

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F

- **If we choose rows (1, 5, 4) then truth value of the minor clause (bvc) <u>remain same</u>** for the active **clause a to be T and F**.
- **But if we choose rows (1, 6, 4) then truth value of the minor clause (bvc) <u>do not remain same</u>** for the active **clause a to be T and F**.

Resolving Ambiguity

$p = a \wedge (b \vee c)$

Major clause: ***a***

Minor clause: $(b \vee c)$

a **= TRUE, *b*=TRUE, *c*=TRUE**

a **= FLASE, *b*=FALSE, *c*=TRUE**

For *a* to be major, (*bvc*) should be TRUE, which can be achieved in 3 combinations of truth values of b and c. So can we design test cases with one combination for a=T and another for a=F?

- This is a confusing question for the test engineers.
- **Considering this leads to three separate criteria**:
  - **GACC: Minor clauses do not need to be the same**
  - **RACC: Minor clause do need to be the same**
  - **CACC: Minor clauses force the predicate to become both true and false**

Types of Active Clause coverage

- **Three kind of active clause criteria** are defined **based on the value that the minor clause take**:
  - **General active clause coverage (GACC)**
  - **Restricted active clause coverage (RACC)**
  - **Correlated active clause coverage (CACC)**

General Active Clause coverage

- General Active Clause Coverage (GACC): For each *p* in P and each major clause c_i in C_p, choose minor clause c_j, $j \neq i$, so that **c_i determines p**.
- TR has two requirements for each c_i :
  - c_i evaluates to true and c_i evaluates to false.
  - The values chosen for the minor clause c_j **do not need to be the same** when c_i is true as when c_i is false.
  - i.e., **$c_j(c_i=T) = c_j(c_i=F)$ for all c_j OR $c_j(c_i=T) \neq c_j(c_i=F)$ for all c_j**

Example of GACC

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F

a is the active clause

bvc is the active clause

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F

- **GACC criteria can be satisfied by taking any test cases from the following nine cases:**
- **(1, 5, 4), (1, 6, 4), (1, 7, 4), (2, 5, 4), (2, 6, 4), (3, 7, 4), (3, 5,4), (3, 6, 4), (3, 7, 4)**

GACC may not subsume predicate coverage

	a	*b*	*p=a ^ b*
1	T	**T**	T
2	T	F	F
3	F	**T**	F
4	F	F	F

In this example, when **b=T**, value of ***p*** depends on the value of ***a.*** But when **b = F**, value of ***p*** **is F** irrespective of the value of ***a.*** Therefore, when ***b*=T, *a* determines *p*. Now let us consider a↔b:**

GACC may not subsume predicate coverage

	a	b	a ↔ b
1	T	T	T
2	T	F	F
3	F	T	F
4	F	F	T

When **b=T**, value of ***p*** depends on the value of ***a.***
But when **b = F**, value of ***p*** **is a' (opposite of a).** Therefore, ***a*** **determines** ***p*** **irrespective of the value of** ***b*****.**

When **a=T**, value of ***p*** depends on the value of ***b.***
But when **a = F**, value of ***p*** **is b' (opposite of b).** Therefore, ***b*** **also determines** ***p*** **irrespective of the value of** ***a*****.**

- If we choose the **test input (1,4)** {a=b=T, a=b=F}, it will **satisfy ACC criteria for both the clauses a and b, without satisfying predicate coverage**.
- **Therefore, GACC is not good criteria for real use.**

Restricted Active Clause Coverage

- **Restricted Active Clause Coverage (RACC)**: For each *p* in P and each major clause c_i in C_p, choose minor clause c_j, $j \neq i$, so that **c_i determines p**.
- TR has two requirements for each c_i :
  - c_i evaluates to true and c_i evaluates to false.
  - The values chosen for the minor clause c_j **must be same when c_i is true as when c_i is false**. That is, **c_j $(c_i=T)$ = c_j $(c_i=F)$ for all c_j**
- This has been a common interpretation by aviation developers

Example of RACC

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F

a is the active clause

bvc is the active clause

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F

- **RACC criteria can be satisfied only by the following set of test cases:**
- **(1, 5, 4), (2, 6, 4), (3, 7, 4)**
- **RACC often leads to infeasible test requirements**
- **There is no logical reason for such restriction**

Correlated Active Clause coverage

- **Correlated Active Clause Coverage (CACC)**: For each *p* in P and each major clause c_i in C_p, choose minor clause c_j, $j \neq i$, so that c_i determines p.
- TR has two requirements for each c_i :
  - c_i evaluates to true and c_i evaluates to false.
  - The values chosen for the **minor clauses** c_j **must cause p to be true for one value of the major clause c_i and false for other**. That is, it is required that $p(c_i=T) \neq p(c_i=F)$.
- It is a more recent interpretation
- Implicitly allows minor clauses to have different values
- Explicitly **satisfies (subsumes) predicate coverage**

Example of CACC

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
5	F	T	T	T	F
6	F	T	F	T	F
7	F	F	T	T	F

a is the active clause

bvc is the active clause

Sl. No.	*a*	*b*	*c*	*b v c*	$a \wedge (b \vee c)$
1	T	T	T	T	T
2	T	T	F	T	T
3	T	F	T	T	T
4	T	F	F	F	F

- **CACC criteria can be satisfied by taking any test cases from the following nine cases:**
- **(1, 5, 4), (1, 6, 4), (1, 7, 4), (2, 5, 4), (2, 6, 4), (3, 7, 4), (3, 5, 4), (3, 6, 4), (3, 7, 4)**

Another example on CACC

	a	b	a ↔ b
1	T	T	T
2	T	F	F
3	F	T	F
4	F	F	T

- Another example p: a≡b. CACC test cases are
  - **a** is the **major** clause: Test set **(1, 3)** {a=T b=T, a=F b=T}
  - **b** is the **major** clause: Test set **(1, 2)** {a=T b=T, a=T b=F}
  - OR merging these two, Test sets are **(1,2,3)** {TT, FT, TF}.

Reference

- https://www.youtube.com/channel/UC97uej-geFTiCmtHiKZavfw/videos
- Introduction to Software Testing (Edition 2)

  by Paul Ammann and Jeff Offutt (https://cs.gmu.edu/~offutt/softwaretest/)